



FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE DE
COIMBRA

Internet of Things

July 2024

IoT BASED SMART BUS-STOP SYSTEM

Report 3 - Final

Professors:

Paulo José Monteiro Peixoto

David Bina Siassipour Portugal

Student:

Stefano Trenti

1 Introduction and Integration Diagram

The project outlined in this report is about the implementation and development of an IoT based smart bus-stop system. The main concept is to provide users information about the live position of incoming busses and estimated time of arrival information to enhance the ease of use of a public transportation system. To realize this project 5 main components are needed: bus stop node, cloud bridge, bus node, cloud infrastructure, GUI. The communication between these components is fundamental to the accomplishment of the task at hand. Given the fact that Amazon AWS will be the cloud platform used for this project and given it's support for MQTT instead of the COAP standard, MQTT will be used as the communication protocol for all the nodes in this project to guarantee continuity and avoid unnecessary translations layers between protocols.

The bus stop node will communicate through serial to a local Mosquitto MQTT broker, then to a Cloud Bridge which will process the local message and relay the information to the Cloud and process the Cloud messages relaying them to the local MQTT network.

The bus nodes will instead communicate directly to the cloud topics to exchange positional, occupancy and stop requests information.

The GUI will instead utilize the AWS Command Line Interface to directly access the data stored in the cloud and display useful information to users.

The AWS Cloud infrastructure will be used to store data, do some processing and communicate between bus nodes and bus stop nodes as well.

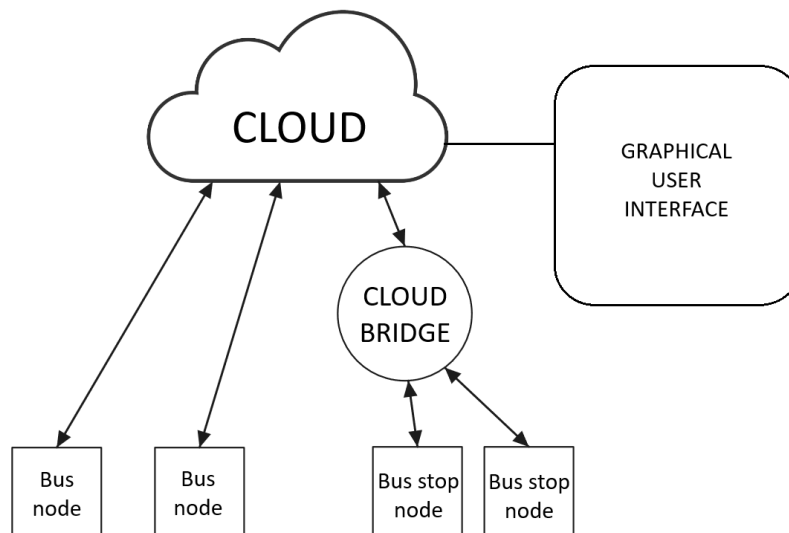


Figure 1: Integration Diagram

2 Bus stop node

To implement the node for a bus stop the Zolertia development board was used. However given the constraints of this board, mainly it's inability of good encryption and transport layer security, to communicate with the AWS cloud a Cloud bridge was implemented. The duty of this python script is that of communicating with the Zolertia board through serial connection (utilizing Tuneslip6) periodically and relay these communications securely to the cloud. To keep things as simple as possible the Zolertia board publishes periodically a message to the "status/stop_1" topic which the cloud bridge is subscribed to. This message contains all information gathered from the sensors and button for requesting a stop, it is then received by the bridge which computes for which bus the stop request is for (the one with the lowest estimated time of arrival) and divides this message in two messages that are encrypted and published in the cloud topics "weather" and "stop_requests" also adding current time information to the messages so to synchronize the timestamps between all nodes once the information is in the cloud.

```

#Create the first JSON message for "stop_requests"
stop_request_message = {
    "stop_id": payload["stop_id"],
    "ts": str(timestamp),
    "bus_24T": stop_24T,
    "bus_34" : stop_34
}

#Create the second JSON message for "weather"
weather_message = {
    "stop_id": payload["stop_id"],
    "ts": str(timestamp),
    "Temperature": payload["Temperature"],
    "Humidity": payload["Humidity"],
    "Pollution": payload["Pollution"],
    "Light": payload["Light"]
}

```

Figure 2: Messages published to the cloud by the bridge

The cloud bridge is also responsible to gather the timing information from that is published by the cloud topic "timings/stop_1" with ETAs for all busses and to communicate them again through serial and MQTT to the Zolertia board. The Zolertia board then receives this information, it displays it in the terminal and when a bus that was requested to stop arrives at the bus stop it turns off the red led which turns on to indicate that a stop request command was issued when pressing the user button. The development platform for this project was the RE-Mote development board:

- Zolertia RE-Mote development board

The final sensor used to measures air quality, temperature, humidity and light are the following:

- Grove - Air Quality Sensor v1.3 , Analog connected to ADC3 port
- Grove - Temperature & Humidity Sensor Pro, Digital connected to ADC1 port
- Grove - Digital Light Sensor, Digital connected to I2C port

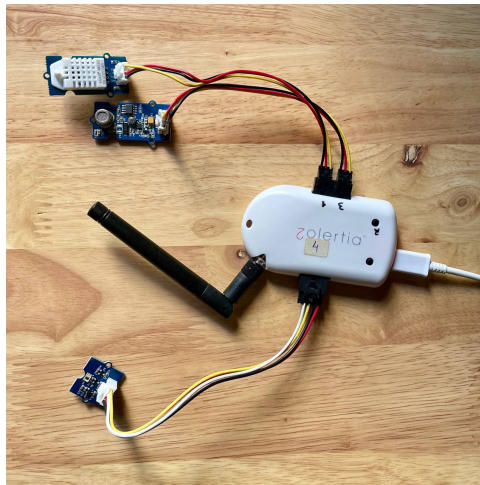


Figure 3: Hardware setup

3 Bus node

The bus node was implemented in a Python script utilizing the "paho.mqtt.client" library, it's purpose is that of simulating a Bus riding on a circular circuit. The ride is a circle of radius 1 Km where the buses travel around at 50 Km/H completing the round trip in about 7.5 minutes. To show the full capabilities of this system two bus nodes were implemented ("34", "24T") that are starting at different points in the circuit. The bus nodes communicate directly and securely to the Cloud subscribing to

the `bus/"bus_id"/stop_rqsts` topic which is published regularly in the cloud MQTT network and is responsible to send a stop request to these nodes. The stop is simulated by letting the bus not move for 5 seconds once it has reached the destination which requested the stop, then a stochastic amount of passengers will exit and enter the bus (only if the bus stops). Location GPS data (latitude, longitude) is collected every 30 seconds and bundled with the number of passengers is then sent to the Cloud on the `busses/loc_occ` topic.

```
message = {
    "bus_id": bus_id,
    "ts": str(timestamp),
    "latitude": f"{latitude:.6f}",
    "longitude": f"{longitude:.6f}",
    "occupancy": str(occupancy)
}
```

Figure 4: Messages published to the cloud by the bus nodes

4 Cloud node - AWS

The Cloud provider of choice for this project was Amazon Web Services or AWS. It has strong MQTT support and provides a flexible and capable environment for this IoT application. The communications between the cloud and the various nodes are handled through the use of DynamoDB tables, that are responsible for ingesting and storing data, and Lambda functions that use the stored data for some computation and publishes messages to the desired topics. As mentioned previously, there are three topics that are used to send information to the Cloud: `weather`, `stop_requests` and `busses/loc_occ`. For each of these topics a DynamoDB table was implemented, this tables take the received JSON messages and save the values of the various variables ordering them by the time-step "ts".

- The *weather_table* is responsible for the ingestion of the messages published by the different bust stop nodes in the `weather` topic. This makes it possible to compute statistics on the weather data coming from all bus stops and makes them accessible to be displayed for the users.
- The *stop_requests_table* instead takes the information from the messages published in the `stop_requests` topic and saves the boolean True/False value corresponding to the stop requests for each bus_ids. A lambda function, called `publish_timings`, is then implemented to gather the latest available information about these variables and then construct and publish a message to the topics `bus/"bus_id"/stop_rqsts` based on the corresponding bus_ids.
- The *location_occupancy_table* takes GPS location and occupancy information coming from the various bus nodes that publish this data on the `busses/loc_occ` topic. This data is then sorted based on the "bus.id" on each message and time stamp. A lambda function was implemented to gather the latest location information and compute estimated time of arrival for each bus and bus stop. The newly created information is then published in the `timings/"stop_id"`.

5 User interface - GUI

The user will have to be able to access information about current position and passengers of busses, and for each bus stop it's corresponding weather data with the computed ETAs for each bus. This can be implemented through a web page, this will make it very easy for users to access the data both from home or from their phones. This GUI - web page was built using HTML, CSS, and JavaScript for the front-end and Python with the Flask framework for the back-end. HTML structures the page, defining sections for bus and weather data. CSS styles these elements, providing layout and visual appearance. JavaScript fetches and dynamically updates the displayed content based on data from the server. The back-end uses Flask to serve the HTML file and provide a REST API endpoint (`/data`) for fetching data. Data is stored in AWS DynamoDB tables, accessed using the Boto3 library. A background thread in Flask periodically updates a global data dictionary by scanning these tables, extracting the latest entries. The front-end periodically fetches this updated data using AJAX calls. This web page can be viewed by

Bus locations and weather conditions are displayed in a box, with pollution levels mapped to descriptive terms ("Fresh Air", "Low", "High", "DANGEROUS") for each bus stop (in this case only one called "COMBATENTES"). Also in each bus stop the estimated time of arrival is displayed for every bus. Bus positions are visually represented by some red dots moving around the circuit with their corresponding labels, the bus stops are instead represented by a blue square. Their passenger information is displayed in separate boxes for each bus fare. The web page fetches new data every couple of seconds through the use of Amazon Command Line Interface which enables the system to access the data stored in all the DynamoDB tables. To safely use this feature a user group with certain permission is created and used as credential information for the CLI.

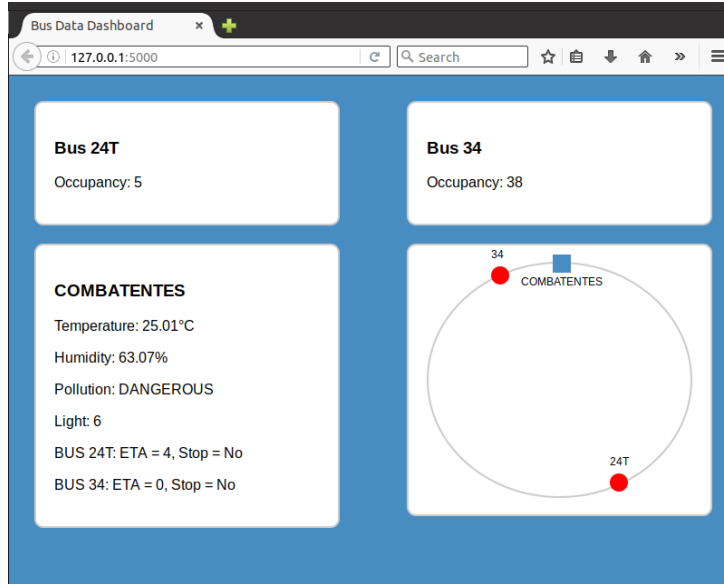


Figure 5: GUI web page

6 Expectations and Conclusions

The desired result of this project is that of creating a smart and interconnected bus transportation system to help provide users a better experience and more convenience. Obtaining real time location of buses, weather data and user occupancy data the aim is that of providing relevant information that can improve the user experience. The result of this project is a working proof of concept that show the capabilities and scalability of this system since the number of bus stops and bus fares is arbitrary and can easily be enlarged to adjust to the size of the bus network. Different methods and solutions were used compared to the previous report to obtain a better working result. MQTT was utilized throughout the whole system to exploit the high integration with the AWS infrastructure and to keep the system's communication protocols consistent. The bus nodes were implemented separately using python scripts instead of running them on the Cooja simulator. A cloud bridge was implemented to connect the local network where the Zolertia board publishes and receives messages to the cloud network instead of a border router and the bus nodes communicate directly to the cloud which is ideal for highly movable nodes. The sensor data acquisition system has shown to be working reliably apart from the air quality sensor that sometimes shows some randomness in the value initialization. The system as a whole has proven to be working quickly and reliably, correctly gathering/sharing/displaying information between all nodes, also the way this system was designed allows it to be easily scalable since everything relies on "bus_id"s and "stop_id"s even though only one bus stop was used to show the system functioning. The raw data obtained from the bus location and weather data coming from the bus stops is now available to be used as statistics and metrics to further improve the optimization of the transportation system and compute weather statistics to improve forecasting.